Divide-and-Conquer Algorithms

Closest Pair of Points and Karatsuba Integer Multiplication

Implementation, Datasets and Experimental Evaluation

Course: Design and Analysis of Algorithms (DAA)

Instructor: Sir Farrukh Salim

Submitted by

[Sana Munir Alam] [24K-0573]

[Sharjeel Memon] [24K-0555]

[Adeena Asif] [24K-0628]

Project Phase 3

Abstract

Divide and conquer breaks a problem into smaller copies of itself, solves those, and stitches the answers together. This project puts the idea to work on two classic problems: finding the closest pair among points in the plane, and multiplying very large integers with Karatsuba's method. Both were written in Python, tested on twenty seeded random datasets (ten per problem, each larger than 100 elements) and checked for correctness against independent references. The closest-pair solver agreed with an exhaustive brute-force search on all ten point files, and the Karatsuba product matched Python's built-in multiplication on all ten integer files, including numbers of 20,000 digits. The measured behaviour follows the expected growth rates of O(n log n) and O(n^1.585) respectively. A desktop interface written in Python, built on the same solver modules, lets the user load any input file and step through the recursion.

1. Introduction

Many problems that look expensive at first glance become tractable once they are cut in half repeatedly. The strategy has three steps: divide the input, conquer each part recursively, and combine the partial results. Its cost is usually described by a recurrence, and the Master Theorem then gives the running time directly [1], [2].

We study two problems where this strategy beats the obvious method. Given n points, the straightforward way to find the nearest two is to test every pair, which costs $\Theta(n^2)$. Shamos and Hoey showed that a divide-and-conquer approach brings this down to $\Theta(n \log n)$ [3], [5]. In the second problem, multiplying two n-digit numbers the school way needs $\Theta(n^2)$ digit operations, whereas Karatsuba and Ofman's trick of replacing four sub-multiplications by three gives $\Theta(n^{\log_2 3}) \approx \Theta(n^{1.585})$ [4].

Our goals were practical as well as theoretical. We wanted working implementations, reproducible datasets, an honest check that the answers are right, and measurements that can be set against the theory.

2. Proposed System

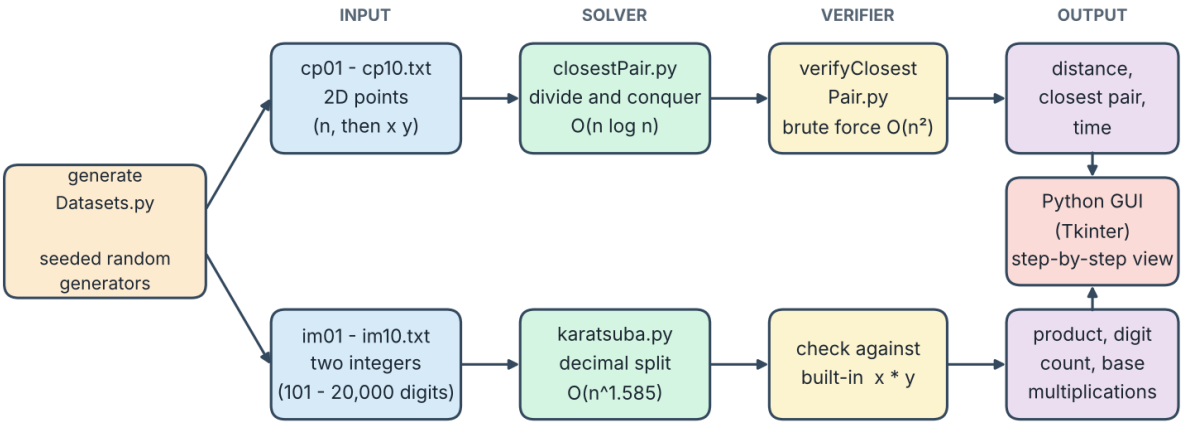


Figure 1. System architecture: dataset generation, solvers, verification and presentation.

Dataset generator. generateDatasets.py uses a separate seeded random generator for every file, so any file can be recreated exactly. It also writes a manifest listing each file's type, size and seed.

Closest pair solver. The points are sorted once by x and once by y. The x-sorted list is split at the median and each half is solved recursively; sets of three points or fewer are settled by brute force. If δ is the smaller of the two half-answers, only points lying within δ of the dividing line can form a better pair. These are scanned in y-order, and each one is compared only with the points that follow it until the vertical gap reaches δ, which happens after a constant number of neighbours (at most seven). The y-sorted list is partitioned between the halves through point indices rather than re-sorted, so the combine step is linear and $T(n) = 2T(n/2) + O(n) = \Theta(n \log n)$. Points that share a coordinate or coincide are handled correctly because of the indices.

Karatsuba solver. Each number is written as $x = a \cdot 10^h + b$ and $y = c \cdot 10^h + d$, with the length padded to an even value so both halves are equal. Instead of computing ac, ad, bc and bd, the algorithm computes ac, bd and (a+b)(c+d), and recovers ad + bc by subtraction. Hence $T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{1.585})$. Signs are removed before recursion and restored at the end. Below 32 digits the recursion stops and an ordinary multiplication is used; this cut-off changes constants, not the growth rate. A counter records how many such base-case calls occur.

Verification. verifyClosestPair.py recomputes every answer by checking all pairs and confirms that the reported pair really occurs in the file. Karatsuba results are compared with the built-in product.

User interface. We built the interface as a Python desktop application using Tkinter, with matplotlib embedded for the plots (Figure 2). Choosing Python over a web page let it import closestPair.py and karatsuba.py directly, so the answer, timing and correctness check on screen come from the same modules that produced our benchmarks and not from a

second implementation. Only the step-by-step view uses a small instrumented replay of each recursion, and the program checks that the replay agrees with the imported solver. A Browse dialog opens any input file, and the program works out whether it holds points or integers. The two views work as follows.

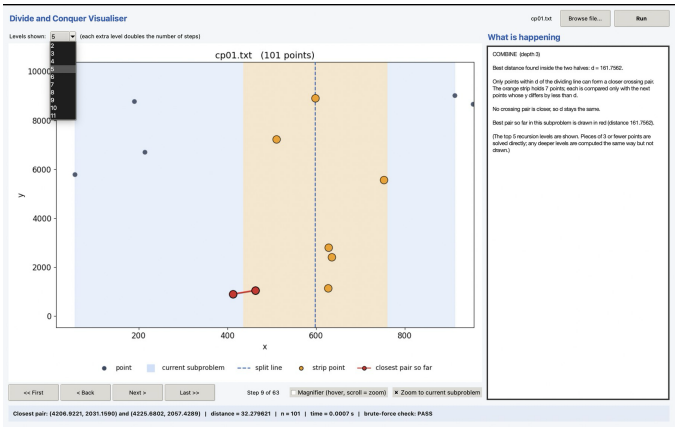
Closest pair view

- **Depth control:** a Levels shown drop-down (2 to 11) sets how many recursion levels are recorded. Each extra level doubles the number of steps, since there are twice as many sub-problems and each has a divide and a combine step (cp01.txt: 3 levels give 15 steps, 5 levels give 63).
- **Plot:** the user steps through the divide and combine stages while the plot highlights the current sub-problem, the split line, the strip of candidate points and the best pair so far, with a plain-language explanation beside it.
- **Limits of the display:** pieces of three or fewer points are solved directly, and deeper levels are computed in the same way but not drawn.
- **Extras:** the last step zooms in on the closest pair, and an optional magnifier helps with dense plots.

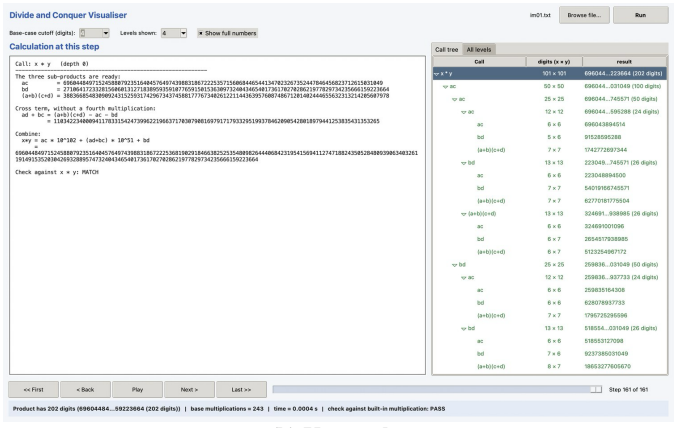
Karatsuba view

- **Controls:** the user sets the base-case cutoff (in digits) and the number of levels shown, and can switch between abbreviated and full numbers.
- **Steps:** the user steps through, or plays, the split into a, b, c and d, the three sub-products, the cross term ad + bc recovered by subtraction without a fourth multiplication, and the final combination, ending with a check against the built-in product.
- **Recursion structure:** a call tree shows the digit sizes and result of every sub-multiplication, next to a per-level table of call counts.

Both views: a status bar reports the result, the running time and the outcome of the check.



(a) Closest pair



(b) Karatsuba

Figure 2. The interface. (a) Closest pair on cp01.txt with the Levels shown drop-down open and set to 5: combine step at depth 3 (step 9 of 63), showing the split line, the strip of seven candidate points and the best pair so far in that sub-problem. (b) Karatsuba on im01.txt with the cutoff set to 4 digits for display, hence 243 base-case calls here against 9 in Table 4, which uses a cutoff of 32.

3. Experimental Setup

All programs use only the Python standard library, and each dataset is a plain text file. A point file starts with the number of points n, followed by one “x y” pair per line with four decimals. An integer file holds two numbers, one per line. Python’s seeded Mersenne Twister generator [6] produced all values, with seeds 1001–1010 for the point files and 2001–2010 for the integer files. Three point distributions were used so the solver is tested on easy and awkward layouts:

Uniform: x and y drawn independently from [0, 10000]. **Clustered:** five centres chosen uniformly in [1000, 9000]², each point picked from a random centre and perturbed by a Gaussian with σ = 150. **Near-collinear:** x uniform in [0, 10000] and y = 0.5x + 100 plus Gaussian noise of σ = 2, which forces many points into a narrow strip. The integers are uniform random values with exactly the stated number of digits (leading digit non-zero), both operands having equal length.

Point file	Distribution	n	Seed	Integer file	Digits	Seed
cp01.txt	Uniform	101	1001	im01.txt	101	2001
cp02.txt	Uniform	500	1002	im02.txt	200	2002
cp03.txt	Uniform	2,000	1003	im03.txt	500	2003
cp04.txt	Uniform	5,000	1004	im04.txt	1,000	2004
cp05.txt	Clustered	200	1005	im05.txt	2,000	2005
cp06.txt	Clustered	1,000	1006	im06.txt	3,000	2006
cp07.txt	Clustered	3,000	1007	im07.txt	5,000	2007
cp08.txt	Collinear	300	1008	im08.txt	8,000	2008
cp09.txt	Collinear	1,500	1009	im09.txt	12,000	2009
cp10.txt	Collinear	4,000	1010	im10.txt	20,000	2010

Table 1. Details of the twenty input files.

Printing all twenty files would fill the report, so Table 2 shows a small sample: the first three lines of cp01.txt and both operands of im01.txt, shortened to their first and last ten digits. The complete data are in the corresponding text files.

Input	Content (sample)
cp01.txt (n = 101)	7966.5097 586.2623 | 7602.3977 8627.0374 | 912.7403 9024.9244 ...
im01.txt, operand 1	9303820964 ... 1638692976 (101 digits)
im01.txt, operand 2	7481279491 ... 9355743889 (101 digits)

Table 2. Representative sample of the generated inputs.

4. Results and Discussion

Tables 3 and 4 list what each program produced. Times are the single-run (closest pair) or best-of-three (Karatsuba) wall-clock values measured on a MacBook Air, in milliseconds. The columns for the closest pair give the distance and the two points found.

File	n	Distance	Closest pair of points	Time (ms)	Brute force
cp01	101	32.279621	(4206.9221, 2031.1590) – (4225.6802, 2057.4289)	0.2	PASS
cp02	500	10.731873	(5490.4834, 7974.3131) – (5500.5165, 7978.1223)	1.2	PASS
cp03	2,000	4.132260	(3685.2714, 1510.1981) – (3685.3693, 1506.0670)	5.5	PASS
cp04	5,000	1.572895	(5510.4200, 5371.0852) – (5511.9673, 5371.3678)	13.3	PASS
cp05	200	4.506810	(5174.3171, 7897.5848) – (5177.8458, 7894.7813)	0.3	PASS
cp06	1,000	0.814604	(2537.9612, 5253.8491) – (2538.6067, 5254.3460)	2.2	PASS
cp07	3,000	0.374818	(5744.1032, 8146.6179) – (5743.7355, 8146.6906)	8.4	PASS
cp08	300	0.691594	(8423.3934, 4312.2674) – (8423.7292, 4311.6628)	0.5	PASS
cp09	1,500	0.250796	(7959.9323, 4078.7888) – (7960.1297, 4078.6341)	2.7	PASS
cp10	4,000	0.080864	(702.6534, 452.3752) – (702.7290, 452.4039)	9.4	PASS

Table 3. Divide-and-conquer closest pair versus exhaustive search.

File	Digits (each)	Product digits	Base mults	Time (ms)	Check
im01	101	202	9	0.0	PASS
im02	200	400	27	0.0	PASS
im03	500	1,000	81	0.1	PASS
im04	1,000	1,999	243	0.4	PASS
im05	2,000	4,000	729	1.3	PASS
im06	3,000	6,000	2,187	3.2	PASS
im07	5,000	9,999	6,561	11.1	PASS
im08	8,000	16,000	6,579	21.2	PASS
im09	12,000	24,000	19,683	38.2	PASS
im10	20,000	40,000	59,049	107.5	PASS

Table 4. Karatsuba multiplication; Check compares the result with Python's built-in product. Base mults is the number of recursive base-case calls (sub-products of at most 32 digits done directly), not the total of digit-level multiplications.

Correctness. Every one of the ten point files gave the same minimum distance as the exhaustive search, and the reported pair was found in the file each time. All ten products equal the built-in result. We also ran a separate randomised stress test (stressTests.py) with duplicate points, points sharing a coordinate, zero and negative operands, and the textbook cut-off of one digit; no failures appeared.

Closest pair. The largest file (5,000 points) took about 13 ms, while an exhaustive search would examine roughly 12.5 million pairs. Running time depends mostly on n and hardly on the layout: the 4,000 near-collinear points were solved in 9.4 ms, which is in line with the 5,000 uniform points at 13.3 ms. Because the files differ in both size and type, we treat the timings as a rough trend rather than a precise fit.

Karatsuba. The number of base-case calls is the clearest evidence of the recurrence. It runs 9, 27, 81, 243, 729, 2,187 as the input roughly doubles, so each doubling triples the leaves, exactly the behaviour of three recursive calls per level; a quadratic method would quadruple the work. The 8,000-digit file shows 6,579 rather than 6,561 because the sums a+b and c+d can carry into one extra digit, pushing a few sub-problems over the cut-off for another split. Run time grows faster than n^1.585 for the largest sizes (about 2.8 times when going from 12,000 to 20,000 digits, against an ideal of 2.3). We attribute this to converting between strings and integers and to multiplying by large powers of ten at each level, but we did not profile it to confirm. Python's own multiplication is written in C, so we used it as a reference answer and not as a speed benchmark.

5. Conclusion

Both divide-and-conquer algorithms were implemented, run on twenty reproducible datasets, and validated against independent checks with no mismatches. The closest-pair solver's speed tracked n log n, and Karatsuba's leaf counts followed powers of three, as the recurrences predict, and the Python interface makes both recursions visible step by step. Three limits are worth stating: timings come from a single machine, the Karatsuba cut-off means our counts are not single-digit multiplications, and the interface draws only the number of recursion levels chosen in its drop-down (at most 11); since each extra level doubles the number of steps, the deepest levels of a large input are impractical to step through.

References

[1] A. Levitin, Introduction to the Design and Analysis of Algorithms, 3rd ed. Pearson, 2012.

[2] J. Kleinberg and É. Tardos, Algorithm Design. Addison-Wesley, 2006.

[3] M. I. Shamos and D. Hoey, "Closest-point problems," in Proc. 16th IEEE Symp. on Foundations of Computer Science (FOCS), 1975, pp. 151–162.

[4] A. Karatsuba and Yu. Ofman, "Multiplication of multidigit numbers on automata," Soviet Physics Doklady, vol. 7, pp. 595–596, 1963.

[5] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed. MIT Press, 2009, Sec. 33.4.

[6] Python Software Foundation, "random — Generate pseudo-random numbers," Python 3 Documentation.